Event Management Portal

Project Report and Technical Documentation

Comprehensive report for an online event planning, booking, and administration platform.

Project Title: Event Management Portal
Document Type: Major Project Report
Prepared For: Academic / Project Submission
Prepared By: Author Name
Date: April 27, 2026

Group Members

S.No.	Name	Roll Number
1	M. MANIDEEP	AP24110010802
2	D. NIKHIL	AP24110010673
3	M. SIVAMANIKANTA	AP24110010763
4	D. KIREETI	AP24110010706
5	K. MALLI	AP24110010651

This report presents the analysis, design, architecture, modules, workflows, and implementation strategy of an online Event Management Portal developed to simplify event discovery, organizer operations, attendee registration, booking, communication, and reporting.

Contents

1 Introduction **1**

2 Problem Statement **1**

3 Objectives of the Project **2**

4 Project Overview and Case Study **2**

5 Scope of the Project **3**

5.1 Included Scope 3

5.2 Excluded or Future Scope 3

6 System Requirements **3**

6.1 Software Requirements 3

6.2 Hardware Requirements 4

6.3 Development Setup 4

7 Feasibility Study **5**

7.1 Technical Feasibility 5

7.2 Economic Feasibility 5

7.3 Operational Feasibility 5

7.4 Schedule Feasibility 5

8 Methodology **5**

9 Technical Architecture **6**

9.1 High-Level Architecture 6

9.2 Architecture Explanation 6

10 Database Design and ER Model **7**

10.1 Primary Entities 7

10.2 Relationships 7

10.3 Illustrative ER View 8

11 Functional Requirements **8**

11.1 Authentication and Authorization 8

11.2 Event Management 8

11.3 Booking Management 8

11.4 Dashboard and Reports 8

12 Non-Functional Requirements **9**

13 Modules and Key Features **9**

13.1 User Registration and Login 9

13.2 Event Creation Module 9

13.3 Event Browsing Module 9

13.4 Booking Module 9

13.5 Admin Dashboard 9

13.6 Vendor Management 10

14 Roles and Responsibilities **10**

14.1 Attendee / User Role 10

14.2 Organizer Role 10

14.3 Admin Role 10

15 User Flow **11**

15.1 Attendee Flow 11

15.2 Organizer Flow 11

15.3 Admin Flow 11

16 MVC Pattern in the Project **11**

16.1 Model Layer 12

16.2 Controller Layer 12

16.3 View / Routing Layer 12

16.4 Advantages of MVC 12

17 Backend Development **12**

17.1 Suggested Backend Structure 12

17.2 Backend Responsibilities 13

17.3 API Design Considerations 13

18 Database Development **13**

18.1 Database Connection 13

18.2 Schema Design 14

18.3 Validation Strategy 14

19 Frontend Development **14**

19.1 Suggested Frontend Structure 14

19.2 Frontend Responsibilities 15

19.3 Reusable Components 15

20 Security Considerations **15**

20.1 Authentication Security 15

20.2 Authorization Security 15

20.3 Input Validation 15

20.4 File Upload Safety 15

21 Testing Strategy **16**

21.1 Unit Testing Targets 16

21.2 Integration Testing Targets 16

21.3 Manual Testing Checklist 16

22 Expected Results and Benefits **16**

23 Limitations **17**

24 Future Enhancements **17**

25 Conclusion **17**

26 Appendix A: Proposed API Endpoints **18**

27 Appendix B: Sample Screen Documentation **18**

28 Appendix C: Where to Add Code **25**

28.1 Frontend Code Placement 25

28.2 Backend Code Placement 26

28.3 How to Present Code in the Report 27

29 Appendix D: Sample Code Snippets **27**

29.1 Sample Database Connection Code 27

29.2 Sample Event Model Code 27

29.3 Sample Route Code 28

29.4 Sample Controller Code 28

29.5 Sample React Component Code 28

30 Appendix E: Setup Summary **29**

30.1 Client Setup 29

30.2 Server Setup 29

30.3 Development Notes 29

1 Introduction

Event Management Portal is a web-based platform developed to manage the lifecycle of events in a centralized and user-friendly manner. The system is intended to support event organizers, administrators, vendors, and attendees by offering features such as account creation, event publishing, ticket booking, scheduling, communication, and reporting. Traditional event coordination often depends on spreadsheets, manual calling, scattered messages, and disconnected tools, which leads to poor visibility, duplication of work, and delays in decision-making.

The main purpose of the portal is to digitize event planning and participation workflows. The application allows organizers to create and manage events, attendees to browse and register for events, and administrators to supervise users, listings, vendors, and activity records. By combining frontend usability with backend data management, the project creates a practical example of a modern full-stack web application.

This report is written as a complete academic-style project document. It explains the problem being addressed, project objectives, scope, requirements, architecture, database design, modules, security, testing, and future enhancements. Space has also been preserved for screenshots so real project images can be inserted later.

2 Problem Statement

Managing events manually is time-consuming and error-prone. Organizers frequently struggle with scattered attendee records, inconsistent scheduling updates, limited communication control, and difficulty in tracking bookings, vendors, and payments. Attendees face their own problems, including poor event visibility, unclear registration processes, and lack of timely updates.

Without a centralized digital solution, the following issues are common:

- event details are stored across multiple disconnected sources,
- booking and attendee registration become difficult to monitor,
- vendor and venue coordination lacks structure,
- notifications and updates are delayed or inconsistent,
- administrators cannot easily oversee the overall system,
- reporting and decision-making are weakened by fragmented data.

The project therefore addresses the need for an integrated Event Management Portal that simplifies event creation, registration, booking, communication, and administration through a secure and scalable web application.

3 Objectives of the Project

The major objectives of Event Management Portal are as follows:

1. To provide secure user registration and login for attendees, organizers, and administrators.
2. To allow organizers to create, update, and publish events.
3. To enable attendees to browse event listings and register or book tickets.
4. To support event categorization, scheduling, and venue management.
5. To provide dashboards for administrators to manage users and monitor activities.
6. To maintain event, booking, vendor, and attendee records in a structured database.
7. To implement the system using modular frontend and backend components.
8. To create a foundation for future features such as payments, analytics, and notifications.

Additional objectives include usability, maintainability, security, and extensibility.

4 Project Overview and Case Study

Consider a scenario involving an event organizer named Riya, who regularly conducts seminars, cultural programs, and workshops. She needs a platform where she can create event listings, define schedules, upload banner images, manage attendee registrations, coordinate vendors, and track participation. At the same time, attendees should be able to discover events, view details, register easily, and receive confirmation.

Using Event Management Portal, Riya can create an event with title, category, venue, date, time, seat capacity, and ticket details. Once published, the event becomes visible to registered users. Attendees can log in, view available events, register or book seats, and track their booking history. Administrators can monitor platform activity, review users, and ensure content quality.

This case study highlights the core value of the system: one platform for planning, management, visibility, and participation. It reduces manual coordination effort and improves user experience for every stakeholder.

5 Scope of the Project

The scope of the portal includes the design and development of a web-based application that handles essential event management operations.

5.1 Included Scope

- User registration and login.
- Organizer event creation and management.
- Attendee event browsing and booking.
- Admin dashboard and user management.
- Venue, category, and schedule information handling.
- Booking history and registration status tracking.
- Vendor or service provider management.
- Backend APIs for CRUD operations.
- Database design using MongoDB and Mongoose.
- Frontend interface using React.

5.2 Excluded or Future Scope

- Production-grade payment gateway integration.
- Real-time messaging and push notifications.
- QR-based entry validation.
- AI-based event recommendation engine.
- Large-scale cloud deployment and multi-tenant administration.

6 System Requirements

6.1 Software Requirements

Component	Purpose
Operating System	Windows 10/11, Linux, or macOS for development and deployment.
Node.js (v16 or above)	Runtime environment for backend logic and frontend tooling.
npm	Package manager for dependency installation and script execution.
React.js / Vite	Frontend library and tooling for responsive interfaces.
Express.js	Backend framework for API development.
MongoDB	NoSQL database for users, events, bookings, vendors, and schedules.
Mongoose	ODM for schema-based data interaction.
Browser	Chrome or Firefox for using and testing the interface.
Visual Studio Code	Preferred editor for development and debugging.
Postman	API testing during development.
Git	Version control for source tracking and collaboration.

6.2 Hardware Requirements

- Processor: Intel i3/Ryzen 3 or above recommended.
- RAM: Minimum 4 GB, 8 GB recommended.
- Storage: At least 2 GB free space for project files and dependencies.
- Internet: Required for package installation and deployment workflows.

6.3 Development Setup

The project follows a full-stack architecture with separate client and server modules. The frontend handles user interaction and visual rendering, while the backend processes requests, applies business logic, and stores data in MongoDB.

7 Feasibility Study

7.1 Technical Feasibility

The project is technically feasible because it is built using widely adopted technologies. React supports a dynamic user interface, Express simplifies backend routing, and MongoDB provides flexible data modeling. These technologies are suitable for student projects and scalable enough for future extension.

7.2 Economic Feasibility

The proposed system is cost-effective because the major development tools are open source. This reduces licensing expenses and makes the project practical for academic use and small-scale implementation.

7.3 Operational Feasibility

The system is operationally feasible because it reflects real event workflows. Organizers can create and monitor events, users can register and book, and administrators can manage data centrally. The learning curve is moderate, making adoption realistic.

7.4 Schedule Feasibility

The application can be implemented in phases, beginning with authentication and event listing, followed by booking, dashboards, and administrative tools. This phased structure makes the project manageable within a typical academic schedule.

8 Methodology

The project can be developed using an iterative methodology:

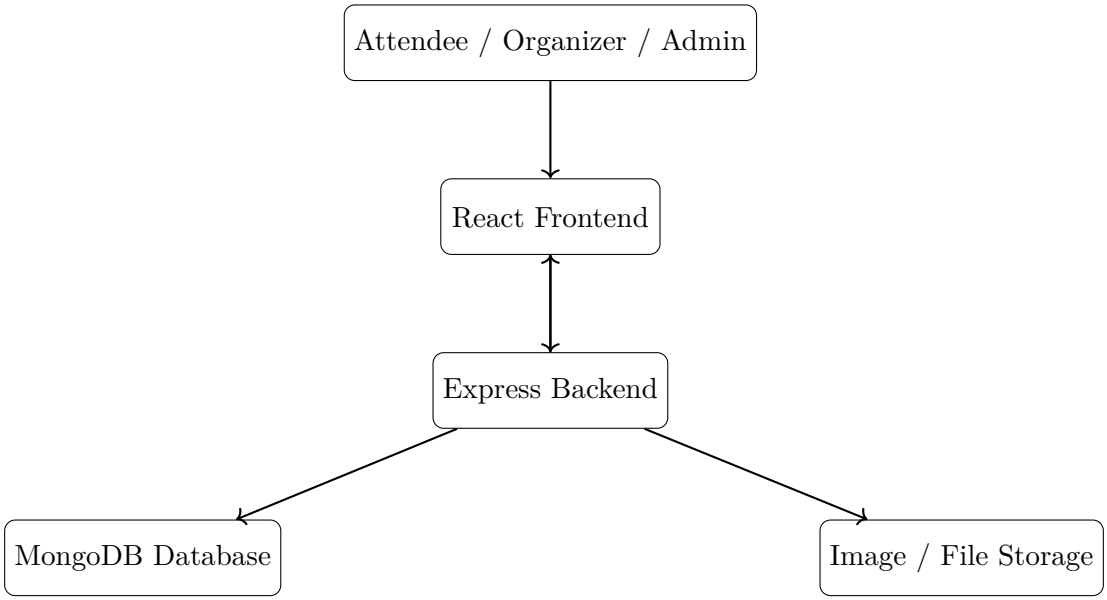
1. Requirement gathering and domain analysis.
2. Identification of user roles and major modules.
3. Design of system architecture and database schema.
4. Backend implementation of models, routes, controllers, and middleware.

5. Frontend implementation of pages and reusable components.
6. API integration between frontend and backend.
7. Testing of registration, event management, and booking workflows.
8. Documentation and presentation preparation.

9 Technical Architecture

The Event Management Portal uses a layered architecture that separates presentation, application logic, and data storage.

9.1 High-Level Architecture



9.2 Architecture Explanation

Users interact with the frontend through forms, dashboards, event cards, and booking pages. The React frontend sends requests to the Express backend, which validates inputs, processes actions, and communicates with the database through Mongoose models. If banners, certificates, or promotional files are included, file storage can also be connected.

This architecture provides:

- clear separation of concerns,

- easier maintenance and scaling,
- role-based interface support,
- modular development,
- simplified testing and debugging.

10 Database Design and ER Model

A strong database structure is essential for handling event records, user details, and bookings.

10.1 Primary Entities

Entity	Primary Key	Important Attributes
User	userId	name, email, password, phone, role, profileStatus
Admin	adminId	name, email, password, permissions
Organizer	organizerId	organizationName, contactPerson, email, phone
Event	eventId	title, description, category, venue, date, time, price, capacity, banner
Booking	bookingId	userId, eventId, seats, totalAmount, bookingStatus, createdAt
Venue	venueId	venueName, address, city, capacity, availability
Vendor	vendorId	vendorName, serviceType, contactInfo, approvalStatus

10.2 Relationships

- One organizer can create many events.
- One user can book many events.
- One event can have many bookings.
- One venue can host many events over time.
- One event can involve multiple vendors or services.

10.3 Illustrative ER View

$$
\begin{tikzcd}
\text{Organizer} \arrow[r, "creates"] & \text{Event} \arrow[r, "has"] \arrow[d, "bookedby"] & \text{Venue} \\
\text{Vendor} \arrow[ur, "supports"'] & \text{Booking} \arrow[l, "for"] & \text{User} \arrow[l, "makes"]
\end{tikzcd}
$$

11 Functional Requirements

11.1 Authentication and Authorization

- The system shall allow users, organizers, and admins to log in securely.
- The system shall restrict access based on role permissions.
- The system shall support protected routes for administrative and organizer tasks.

11.2 Event Management

- Organizers shall be able to create, edit, publish, and delete events.
- The system shall store event details including venue, schedule, category, and seat limit.
- Attendees shall be able to browse and filter events.

11.3 Booking Management

- Users shall be able to register or book tickets for available events.
- The system shall store seat count, booking status, and booking history.
- Organizers and admins shall be able to monitor booking activity.

11.4 Dashboard and Reports

- Admins shall view platform-level summaries.
- Organizers shall view event-specific performance and registrations.

12 Non-Functional Requirements

- **Usability:** the interface should be easy to understand for first-time users.
- **Security:** passwords and sensitive data must be protected.
- **Performance:** event listing and booking operations should respond quickly.
- **Reliability:** the system should reduce duplicate bookings and inconsistent records.
- **Maintainability:** the codebase should remain modular and readable.
- **Scalability:** the structure should support future growth in events and users.

13 Modules and Key Features

13.1 User Registration and Login

This module provides sign-up, login, and account management for attendees and organizers.

13.2 Event Creation Module

Organizers can define event details, schedules, pricing, seat limits, and promotional banners.

13.3 Event Browsing Module

Users can search, filter, and review available events based on category, date, venue, or organizer.

13.4 Booking Module

This module allows users to register or reserve seats and review confirmation details.

13.5 Admin Dashboard

The dashboard helps administrators manage users, events, reported issues, and system-wide statistics.

13.6 Vendor Management

The portal can store vendor information such as catering, decoration, sound, photography, and venue support services.

14 Roles and Responsibilities

14.1 Attendee / User Role

- Register and manage account details.
- Browse available events.
- Book tickets or register for participation.
- View booking history and event details.
- Receive updates and provide feedback.

14.2 Organizer Role

- Create and update event listings.
- Manage schedules, seats, venue, and banner content.
- Track attendee registrations and booking status.
- Coordinate associated vendors or service providers.

14.3 Admin Role

- Monitor users and organizers.
- Approve, edit, or remove inappropriate content.
- Supervise bookings and platform activity.
- Maintain security and configuration settings.

15 User Flow

15.1 Attendee Flow

1. The user opens the home page.
2. The user signs up or logs in.
3. The user browses events and filters listings.
4. The user opens a specific event page.
5. The user books seats or registers.
6. The user views booking confirmation and history.

15.2 Organizer Flow

1. The organizer logs in.
2. The organizer creates or edits an event.
3. The event is published.
4. The organizer tracks attendee registrations.
5. The organizer updates event status or schedule when required.

15.3 Admin Flow

1. The admin logs in through the control panel.
2. The admin reviews users, organizers, and events.
3. The admin monitors reports, bookings, and overall activity.
4. The admin maintains data quality and platform compliance.

16 MVC Pattern in the Project

The backend can be organized using the Model-View-Controller pattern.

16.1 Model Layer

The model layer defines entities such as users, organizers, events, bookings, venues, and vendors using Mongoose schemas.

16.2 Controller Layer

Controllers manage business logic such as registration, event publishing, booking creation, attendee listing, and admin operations.

16.3 View / Routing Layer

In a REST API, route files act as the request-handling layer by mapping HTTP methods to controller functions.

16.4 Advantages of MVC

- modular development,
- improved readability,
- easier maintenance,
- better testability,
- clearer separation of responsibility.

17 Backend Development

17.1 Suggested Backend Structure

```
server/
  config/
    db.js
  controllers/
    adminController.js
    authController.js
    eventController.js
```

```
    bookingController.js
  middleware/
    authMiddleware.js
    upload.js
  models/
    User.js
    Organizer.js
    Event.js
    Booking.js
    Venue.js
    Vendor.js
  routes/
    adminRoutes.js
    authRoutes.js
    eventRoutes.js
    bookingRoutes.js
  server.js
```

17.2 Backend Responsibilities

The backend validates requests, stores event data, processes bookings, controls authorization, and responds to client actions through REST APIs.

17.3 API Design Considerations

A well-designed API should use meaningful status codes, consistent response structures, and proper validation. Common status codes include `200`, `201`, `400`, `401`, `404`, and `500`.

18 Database Development

18.1 Database Connection

MongoDB is connected through a configuration module, and Mongoose models are loaded before processing requests.

18.2 Schema Design

Schemas should include validation for required fields such as event title, date, venue, user email, booking quantity, and role type.

18.3 Validation Strategy

Validation can be handled at schema level, route level, controller level, and frontend form level to improve both integrity and user experience.

19 Frontend Development

19.1 Suggested Frontend Structure

```
client/src/
  Admin/
    Dashboard.jsx
    Users.jsx
    Events.jsx
    Reports.jsx
  Organizer/
    CreateEvent.jsx
    ManageEvents.jsx
    Bookings.jsx
  User/
    Home.jsx
    EventList.jsx
    EventDetails.jsx
    MyBookings.jsx
    Login.jsx
    Signup.jsx
  Components/
    Navbar.jsx
    Footer.jsx
    EventCard.jsx
    SearchBar.jsx
```

19.2 Frontend Responsibilities

The frontend renders forms, event cards, dashboards, filters, booking pages, and profile screens. It communicates with backend APIs and presents data in a user-friendly way.

19.3 Reusable Components

Reusable components such as cards, tables, forms, buttons, and navigation bars help maintain consistency across the project.

20 Security Considerations

20.1 Authentication Security

Passwords should be hashed before storage, and authenticated sessions or JWT-based authorization should be used.

20.2 Authorization Security

Only authorized organizers and admins should be allowed to publish events or manage sensitive data.

20.3 Input Validation

User and organizer input must be validated to prevent malformed or harmful data from entering the system.

20.4 File Upload Safety

Banner or gallery uploads should be limited by type and size, and stored in a controlled location.

21 Testing Strategy

21.1 Unit Testing Targets

Unit testing can cover helper utilities, schema validation rules, and smaller controller functions.

21.2 Integration Testing Targets

Integration testing can validate:

- registration and login,
- event creation,
- event listing retrieval,
- booking creation,
- protected route behavior.

21.3 Manual Testing Checklist

1. Verify signup and login for multiple roles.
2. Verify event creation and editing.
3. Verify event browsing and filtering.
4. Verify booking submission and history.
5. Verify admin monitoring features.
6. Verify upload validation and error handling.

22 Expected Results and Benefits

The expected result is a working portal that supports event publishing, booking, administrative monitoring, and centralized record management. The system improves visibility, reduces manual coordination effort, and offers a better experience to organizers and attendees.

From an academic perspective, the project demonstrates full-stack development skills, including interface design, API construction, database modeling, security awareness, and software documentation.

23 Limitations

- Payment processing may be simulated.
- Real-time messaging may not be implemented.
- Advanced analytics may remain basic.
- High-scale deployment concerns may be outside current scope.

24 Future Enhancements

1. Add payment gateway integration.
2. Add email and SMS notifications.
3. Add QR-code ticket validation.
4. Add analytics dashboards for event performance.
5. Add recommendation and personalized event discovery.
6. Add certificate generation for workshops and seminars.
7. Add mobile app support.

25 Conclusion

Event Management Portal is a practical full-stack application that addresses real challenges in event planning and participation. By combining React, Express, MongoDB, and a role-based workflow, the project creates a structured and scalable system for event creation, registration, booking, and administration.

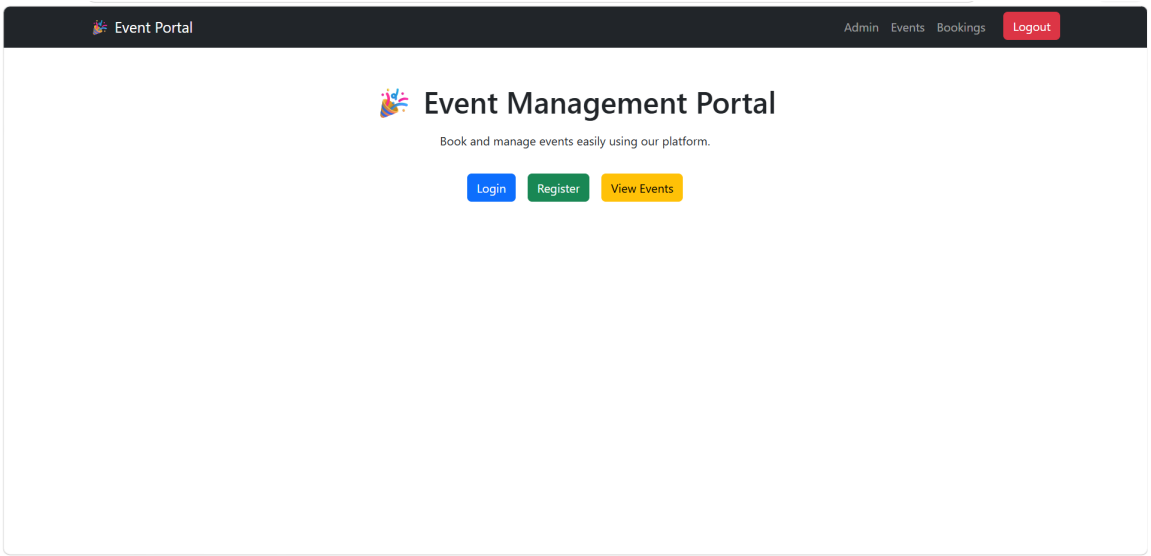
The report presents the problem background, objectives, architecture, modules, security, testing, and future direction of the system in a clear academic format. It can be used directly as a project report and can be further personalized with institution details, team member names, and actual screenshots.

26 Appendix A: Proposed API Endpoints

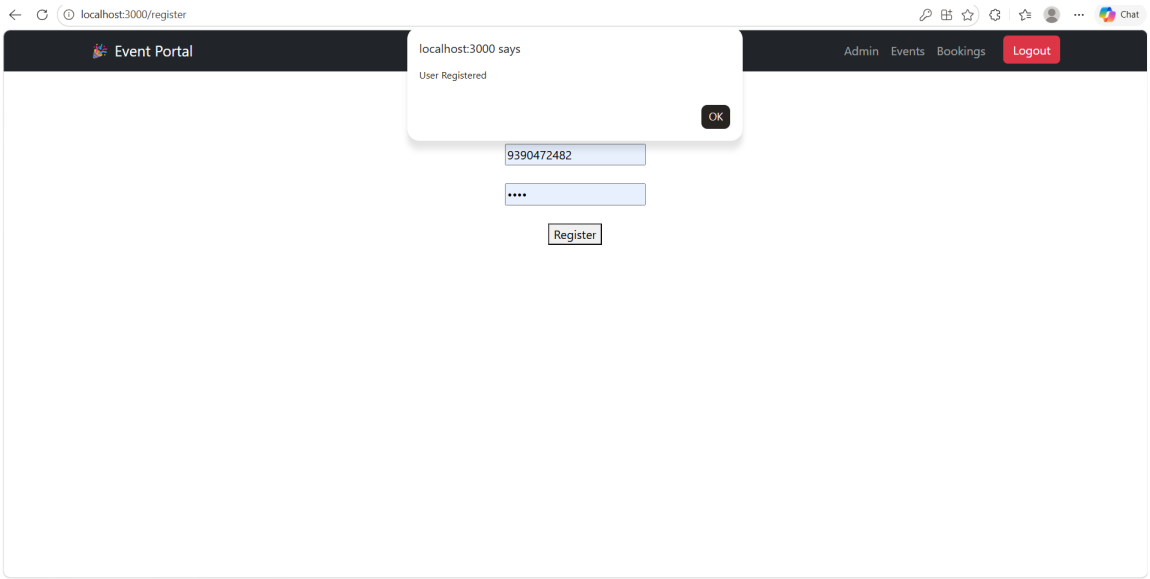
Method	Route	Purpose
POST	/api/auth/register	Register a new user or organizer.
POST	/api/auth/login	Authenticate a user.
GET	/api/events	Fetch all published events.
GET	/api/events/:id	Fetch a specific event.
POST	/api/events	Create a new event.
PUT	/api/events/:id	Update an event.
DELETE	/api/events/:id	Delete an event.
POST	/api/bookings	Create a new booking.
GET	/api/bookings/:id	Retrieve booking details.
GET	/api/admin/users	List all users.
GET	/api/admin/events	Review platform events.
GET	/api/admin/bookings	Review all bookings.

27 Appendix B: Sample Screen Documentation

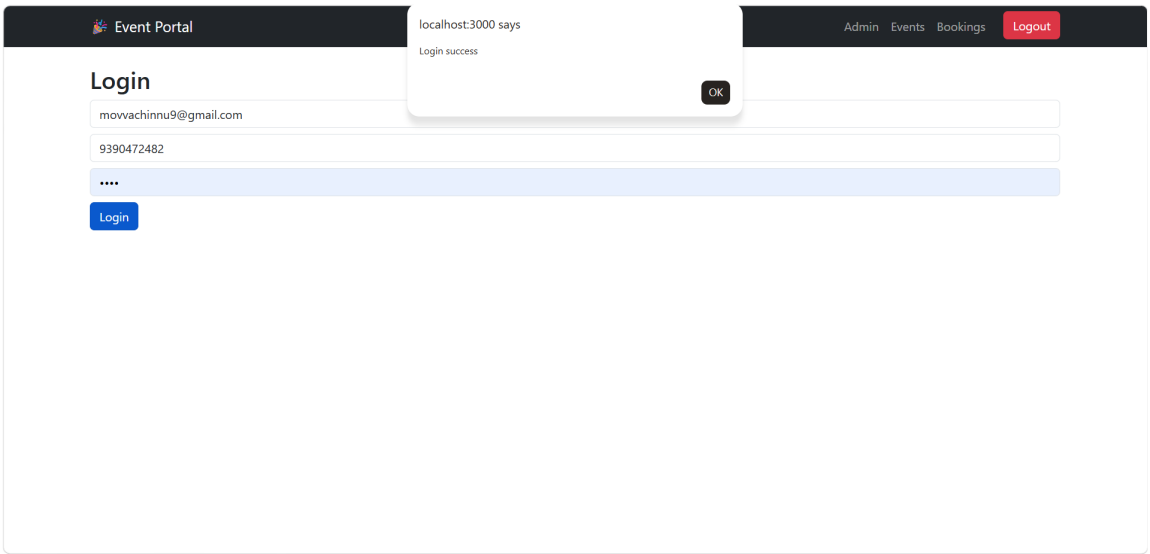
The following placeholders preserve space for project images. You can replace them later with real screenshots from your portal.



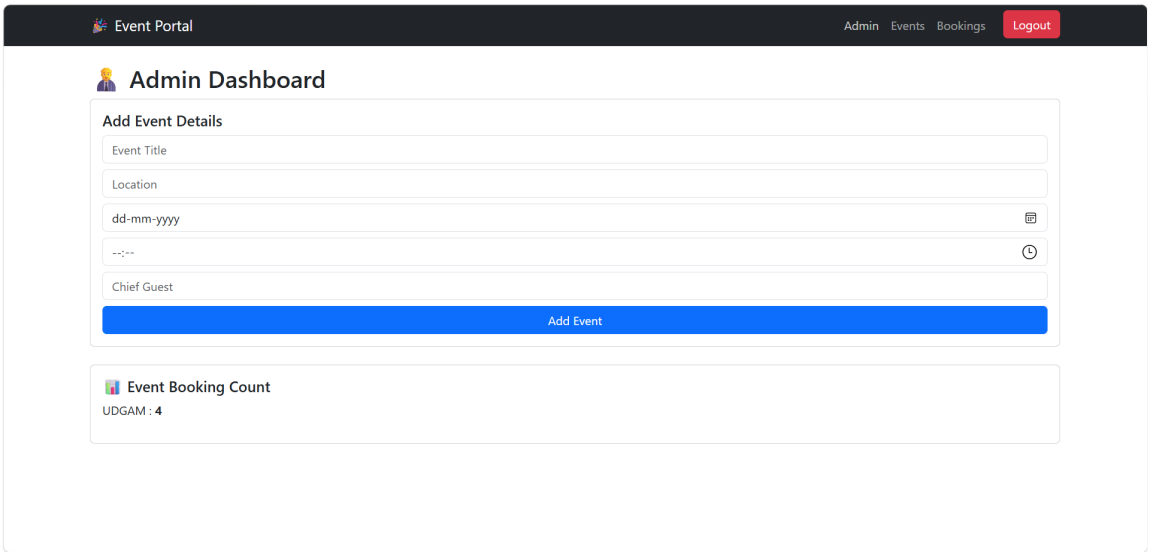
Landing Page



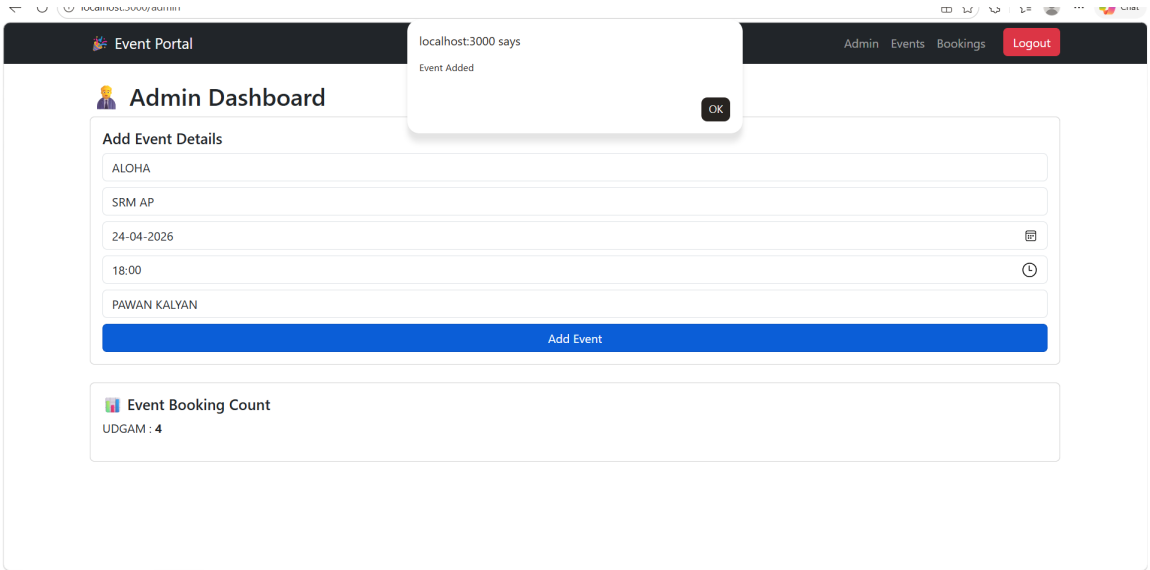
User Registration Page



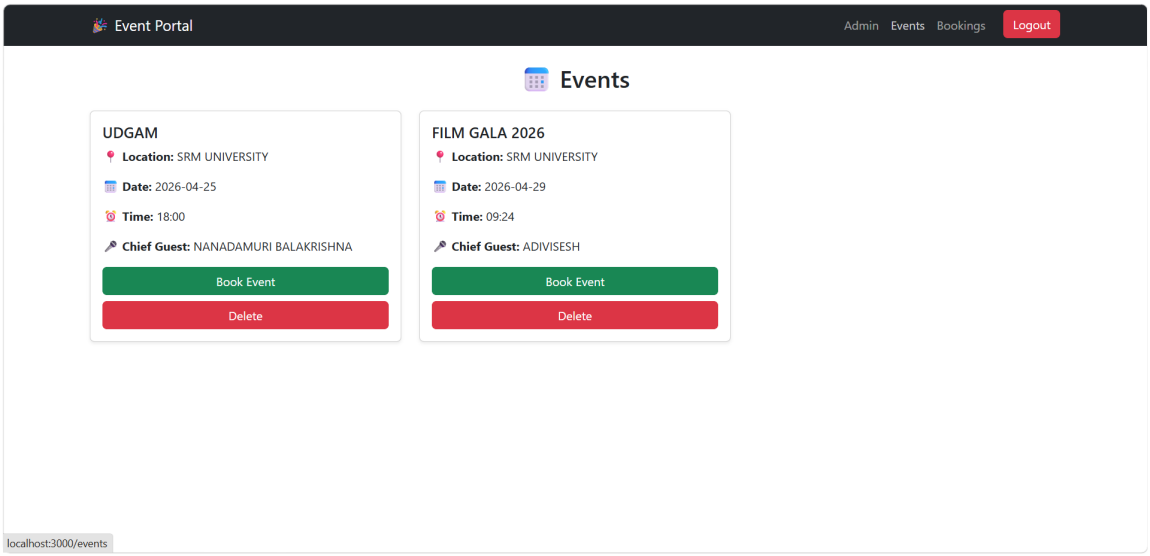
User Login Page



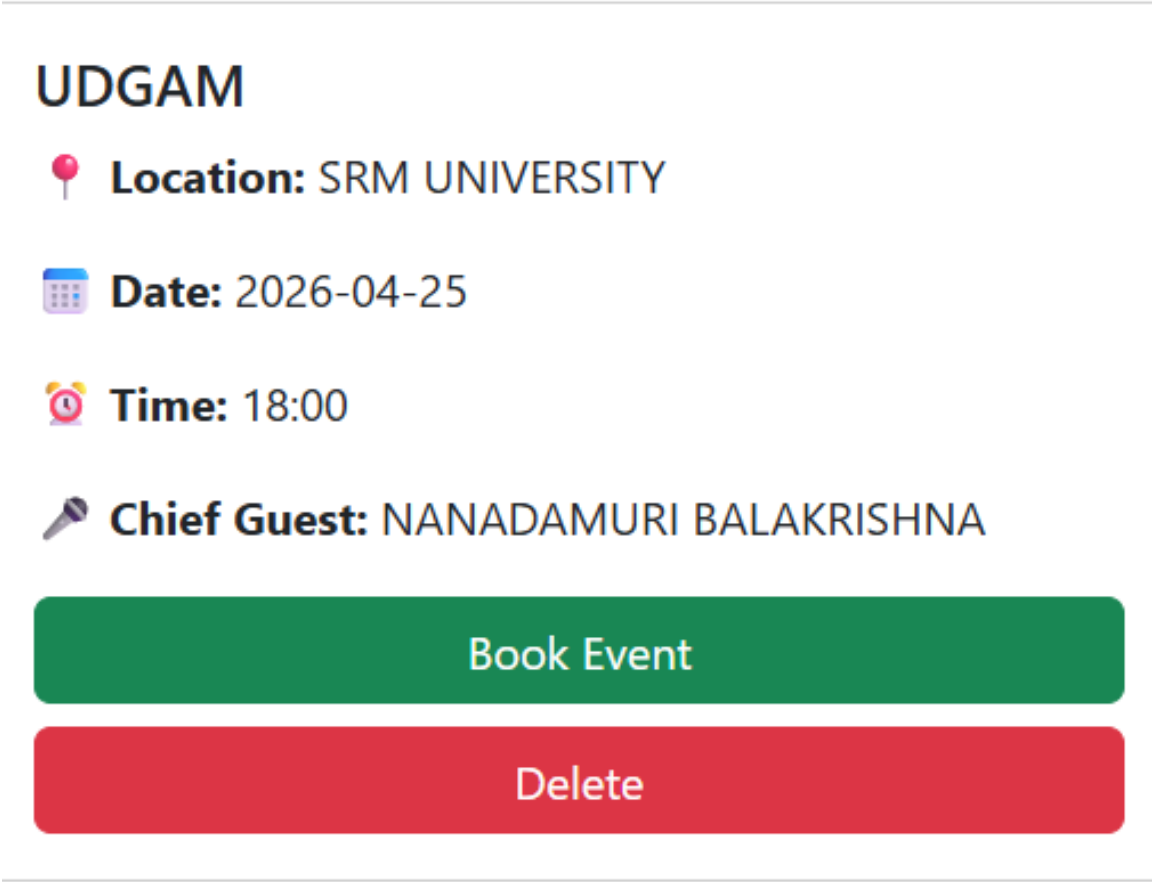
Organizer Dashboard



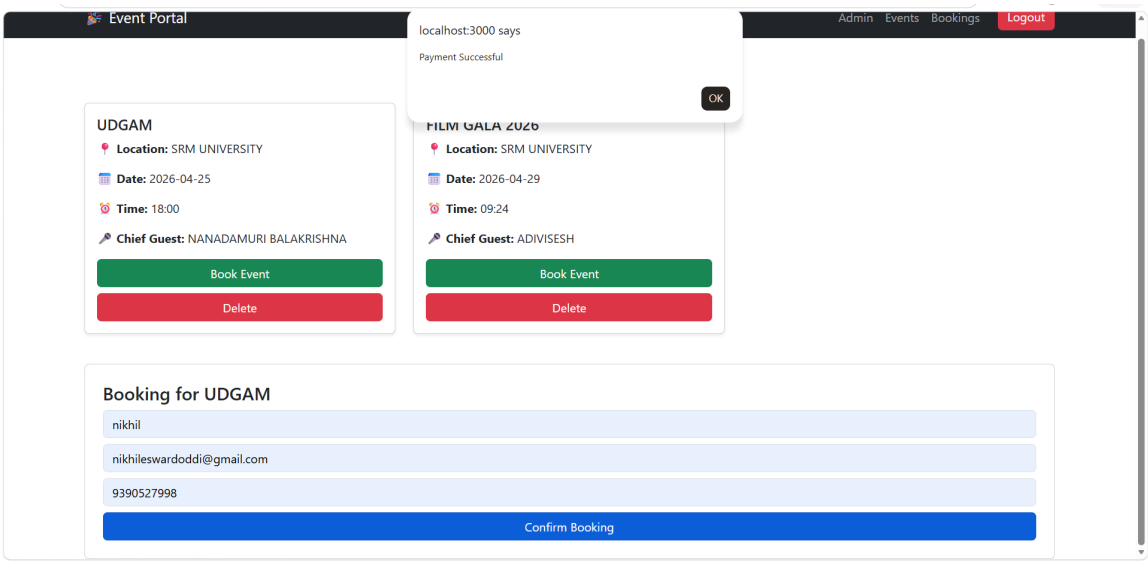
Create Event Page



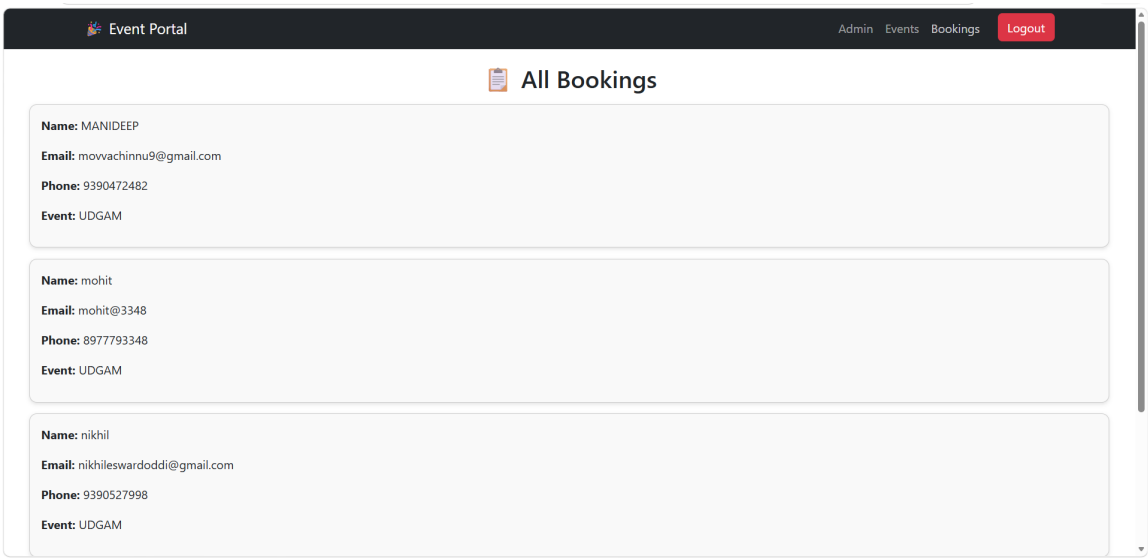
Event Listing Page



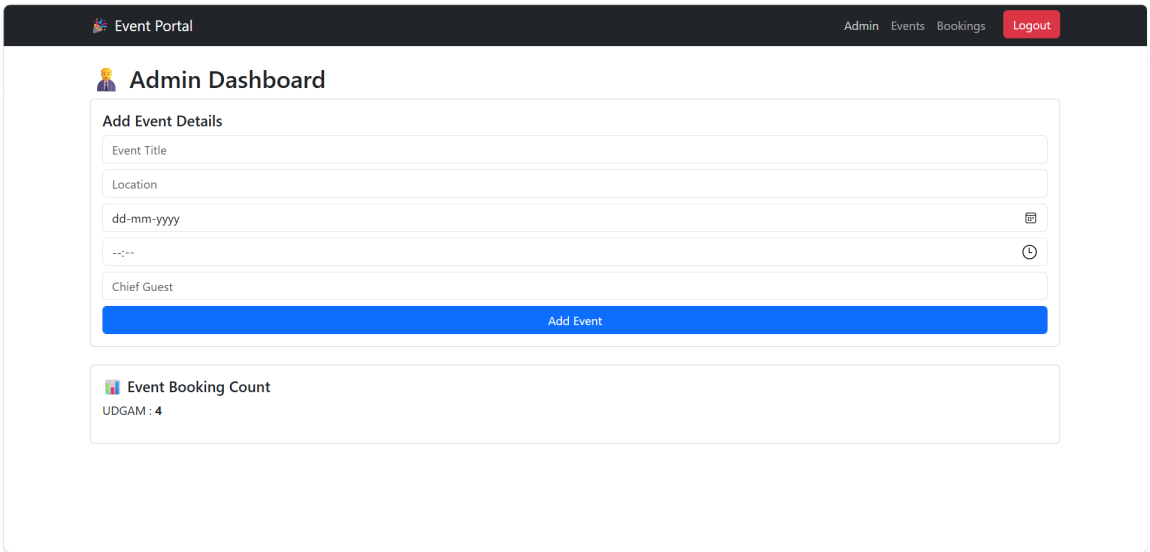
Event Details Page



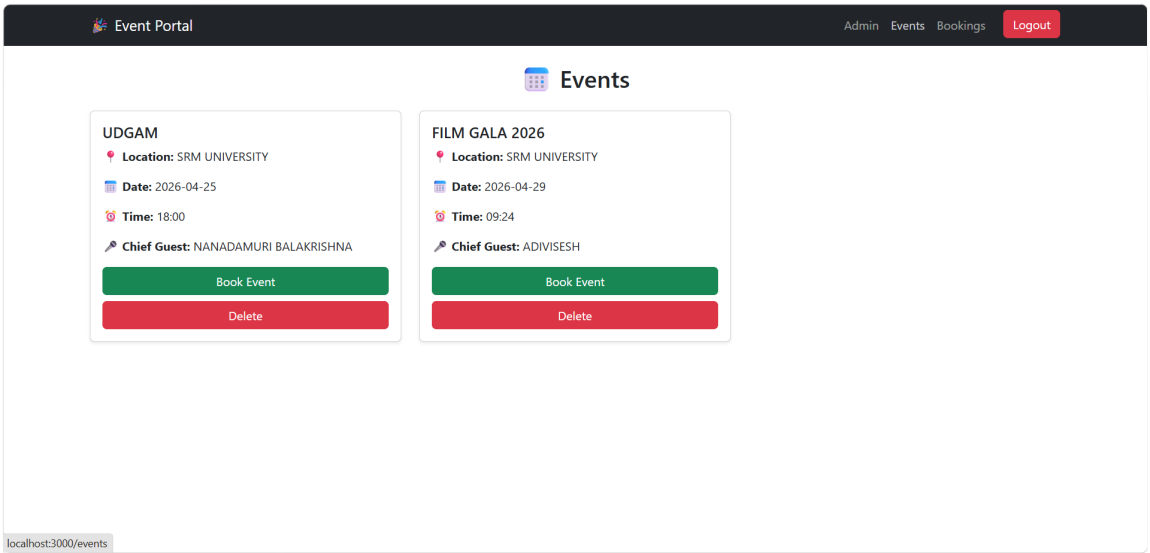
Booking Page



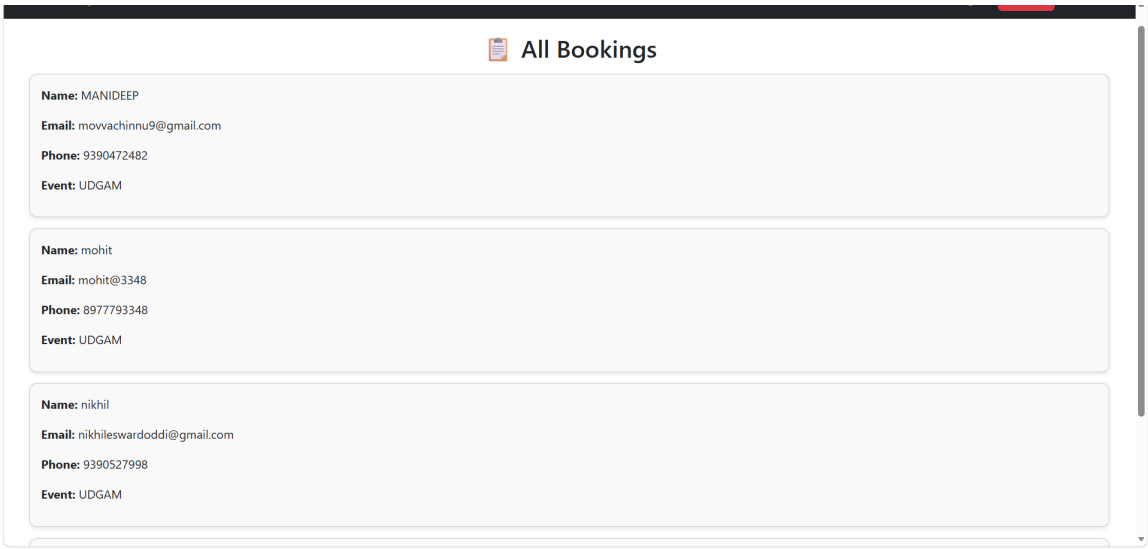
My Bookings Page



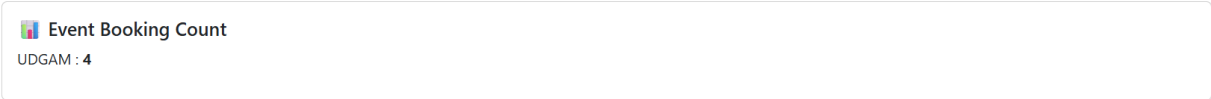
Admin Dashboard



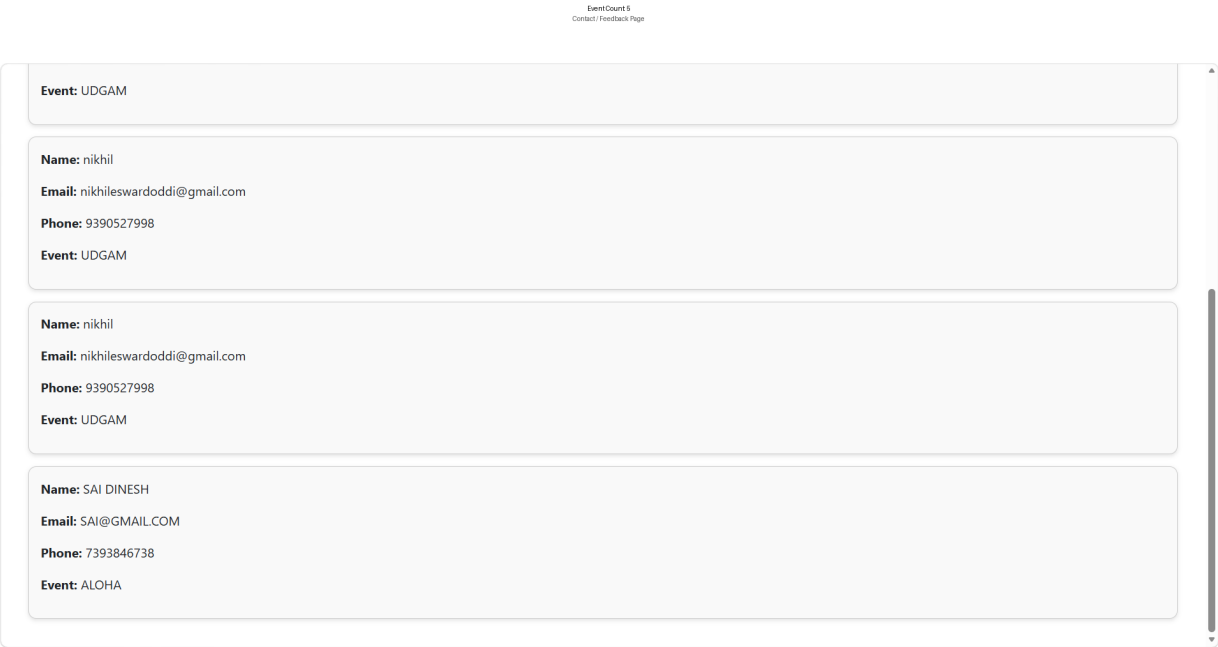
Manage Events Page



Reports or Analytics Page



Reports or Analytics Page (Additional View)



Contact / Feedback Page

28 Appendix C: Where to Add Code

This section explains where project code should be placed in a typical Event Management Portal structure. If you are preparing a final submission, you can keep the explanation in the report and place the actual source code in your project folders or in a separate appendix.

28.1 Frontend Code Placement

Frontend code should be placed inside the client-side source folder. A recommended layout is shown below.

```
client/src/
  Admin/
    Dashboard.jsx
    Users.jsx
    Events.jsx
    Reports.jsx
  Organizer/
    CreateEvent.jsx
    ManageEvents.jsx
    Bookings.jsx
  User/
    Home.jsx
    EventList.jsx
    EventDetails.jsx
    MyBookings.jsx
    Login.jsx
    Signup.jsx
  Components/
    Navbar.jsx
    Footer.jsx
    EventCard.jsx
    SearchBar.jsx
```

- Add login and signup code in files such as `Login.jsx` and `Signup.jsx`.
- Add event listing and event detail UI code in `EventList.jsx` and `EventDetails.jsx`.
- Add organizer-specific forms in `CreateEvent.jsx` and `ManageEvents.jsx`.

- Add shared UI code such as cards and navigation inside the `Components` folder.

28.2 Backend Code Placement

Backend code should be organized into configuration, models, controllers, middleware, and routes.

```
server/
  config/
    db.js
  controllers/
    authController.js
    eventController.js
    bookingController.js
    adminController.js
  middleware/
    authMiddleware.js
    upload.js
  models/
    User.js
    Organizer.js
    Event.js
    Booking.js
    Venue.js
    Vendor.js
  routes/
    authRoutes.js
    eventRoutes.js
    bookingRoutes.js
    adminRoutes.js
  server.js
```

- Add MongoDB connection code in `config/db.js`.
- Add schema definitions in the `models` folder.
- Add business logic in the `controllers` folder.
- Add route declarations in the `routes` folder.
- Add token verification or authentication guard logic in `middleware/authMiddleware.js`.

28.3 How to Present Code in the Report

If your college or guide asks you to include code in the report itself, the best place is usually at the end in an appendix. Include only important code snippets such as database connection, one model, one controller, one route, and one frontend component. Do not paste the entire project source into the main chapters because it makes the document difficult to read.

29 Appendix D: Sample Code Snippets

The following sample snippets show the type of code that can be included in the report. You can replace these with your actual project code later.

29.1 Sample Database Connection Code

```
import mongoose from 'mongoose';

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('Database connected successfully');
  } catch (error) {
    console.error('Database connection failed:', error.message);
    process.exit(1);
  }
};

export default connectDB;
```

29.2 Sample Event Model Code

```
import mongoose from 'mongoose';

const eventSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String, required: true },
  category: { type: String, required: true },
  venue: { type: String, required: true },
  date: { type: String, required: true },
  time: { type: String, required: true },
  price: { type: Number, default: 0 },
```

```
  capacity: { type: Number, required: true },
  banner: { type: String }
}, { timestamps: true });

export default mongoose.model('Event', eventSchema);
```

29.3 Sample Route Code

```
import express from 'express';
import { createEvent, getEvents } from '../controllers/eventController.js';
import authMiddleware from '../middleware/authMiddleware.js';

const router = express.Router();

router.get('/', getEvents);
router.post('/', authMiddleware, createEvent);

export default router;
```

29.4 Sample Controller Code

```
export const createEvent = async (req, res) => {
  try {
    const event = await Event.create(req.body);
    res.status(201).json({ success: true, data: event });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};
```

29.5 Sample React Component Code

```
import React from 'react';

const EventCard = ({ event }) => {
  return (
    <div className="event-card">
      <h3>{event.title}</h3>
      <p>{event.venue}</p>
      <p>{event.date} | {event.time}</p>
```

```
      <button>View Details</button>
    </div>
  );
};

export default EventCard;
```

30 Appendix E: Setup Summary

30.1 Client Setup

```
npm create vite@latest client -- --template react
cd client
npm install
npm run dev
```

30.2 Server Setup

```
mkdir server
cd server
npm init -y
npm install express mongoose cors dotenv bcrypt jsonwebtoken multer
node server.js
```

30.3 Development Notes

Environment variables can be managed in a `.env` file, and API base URLs can be configured separately for development and production.